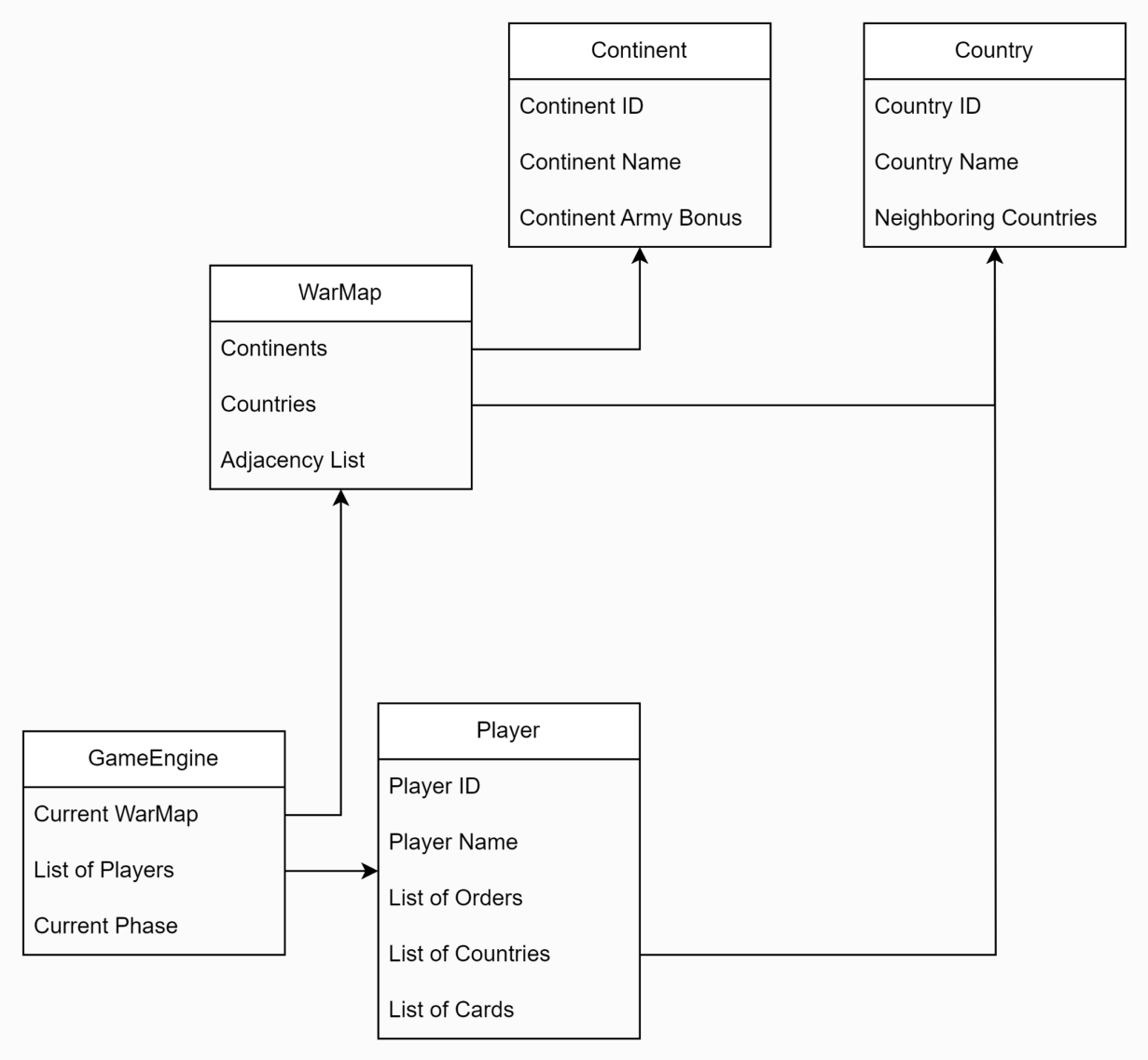
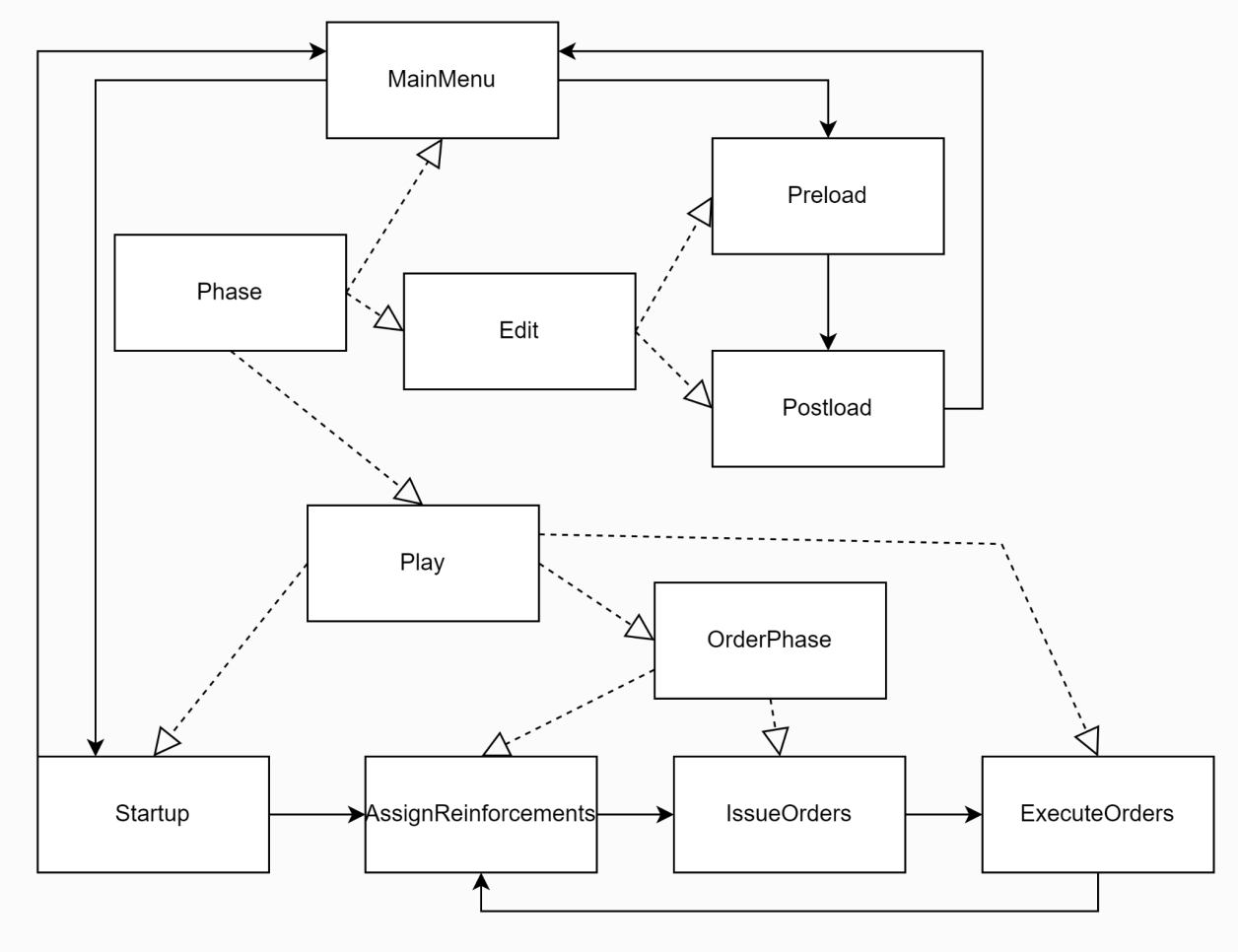
Architectural Design

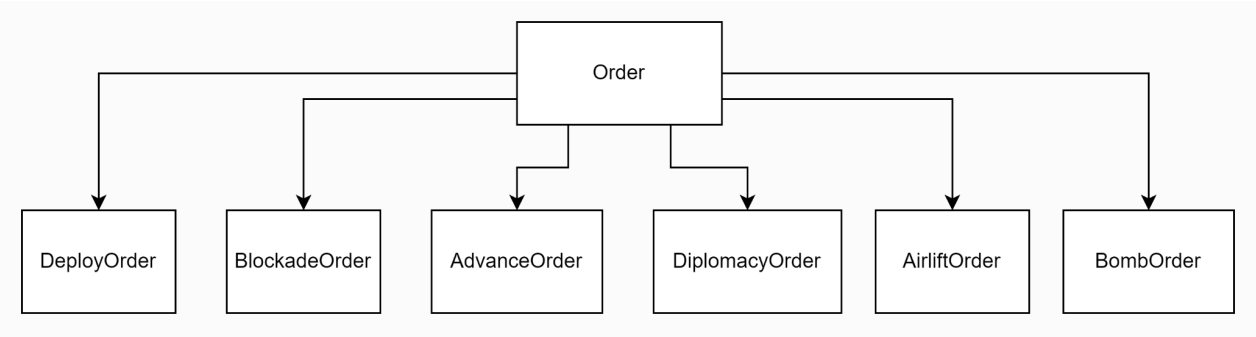
The overarching design of our program is that the game is entirely controlled by the GameEngine, this is done with the GameEngine holding control of the current WarMap, the list of players, and the current phase of the game. The WarMap contains the list of continents, the list of countries and the adjacency List, where the continent is given an ID, name and army bonus and each country is given an ID, name and list of Neighboring countries. The list of players contains players who have an ID, name, list of countries, list of orders, and list of cards. This main structure is shown below.

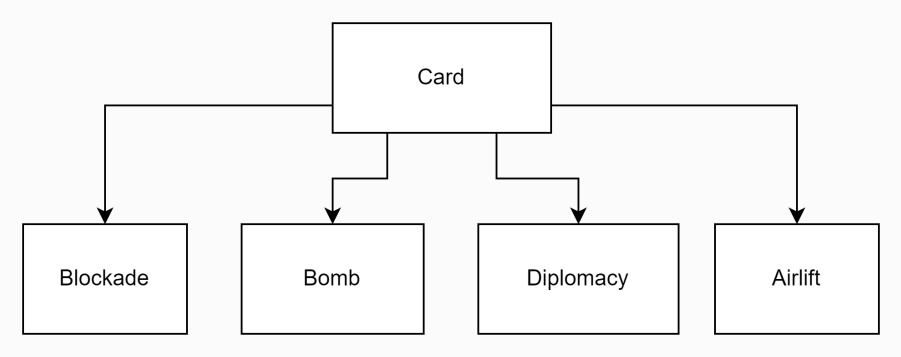


The GameEngine's phase makes use of the state pattern where the MainMenu passes control to either the Edit or Play phases, where the Edit phase is responsible for map editing and the Play phase is responsible for gameplay. The edit phase is separated into preload and postload depending on whether a map has been loaded for editing. The Play phase is separated into Startup, AssignReinforcements, IssueOrders, and ExecuteOrders. In the play phase startup occurs once and the remaining three phases run in a loop, the inheritance of phases is shown with dotted lines and possible flow of phase is shown with solid lines in the following diagram:

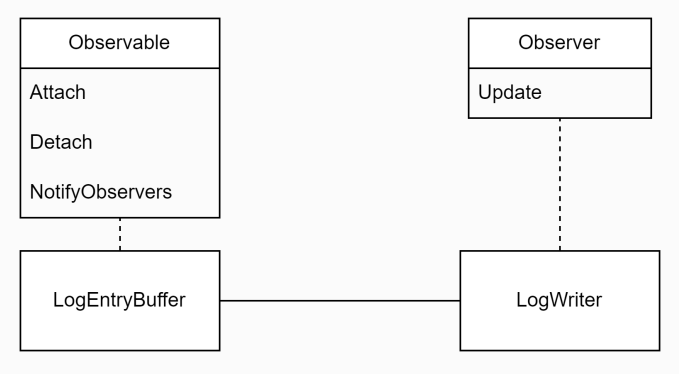


Orders utilize the command pattern by allowing players to hold a list of order's which can be of many different order types. This allows for a singular issue_order() function to be used to create any type of order, the order created depending on user input with the possible types being deploy, advance, airlift, bomb, barricade and diplomacy. Similarly cards are held by players in a list of cards, but are also separated into four different types, that being airlift, bomb, barricade or diplomacy. These are shown in the two following diagrams.

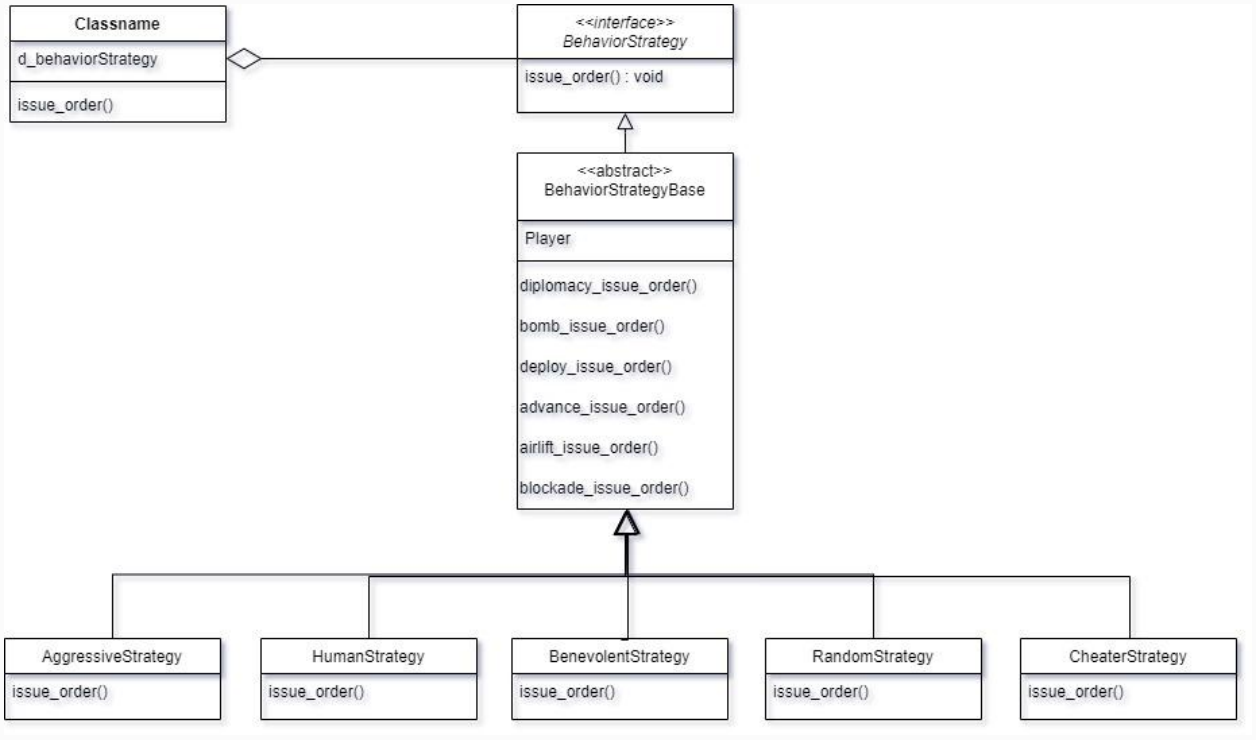




The logging functionality of our program makes use of the observer pattern, the LogEntryBuffer functions as our concrete observable and the LogWriter functions as our concrete observer. Other portions of the program are able to send log messages to the LogEntryBuffer which notifies the LogWriter that it has received a new log message that then calls its update function to write the new message to the log. A diagram of the main relationship of this functionality is shown below.



To implement the different behavior strategies, a Strategy Pattern is used. The issue_order() method of a player calls the issue_order() from the BehaviourStrategy interface which has the specific behavior object attached to it. The behavior is set when creating the players in single player mode. The control is passed to the issue_order() of the specific behavior which then issues deploy or other orders according to the Phase that the game is currently in.



To adapt the current functionality of saving, loading and editing the map files to the newly introduced conquest files, we had to implement an Adapter Pattern over the current MapEditor. This allows us to read and write from the conquest files without the GameEngine class worrying much about that. For this we have created a MapEditorAdapter that extends on the current MapEditor and then delegates the method to the passed in MapEditorConquest to execute that specific behavior.